Canggu: A Quantum-Secure, MEV-Resistant Blockchain for Parallel Computing

Version 2.0 — Polkadot SDK Edition

Original design (v1.3):
Dendi Suhubdy, Tejveer Singh, Mitul Dhawan, Maldini Sonkeng
Bitwyre [21]

Polkadot SDK edition (v2.0):
written by Claude (Anthropic), directed by agustinustheo
https://github.com/agustinustheo/canggu-blockchain

October 31, 2026

Abstract

Canggu is a layer-1 blockchain designed around four ideas: post-quantum account security, a dependency-ordered (DAG) mempool that removes the cheapest forms of transaction reordering, succinct zero-knowledge verification, and execution models that can exploit parallel hardware. Version 1.3 of this whitepaper described those ideas as a from-scratch system. This version specifies how Canggu is built on the Polkadot SDK (Substrate/FRAME), documents a working reference implementation, and revisits the original design with measurements and a critical eye. Concretely we implement (i) ML-DSA-65 (CRYSTALS-Dilithium, FIPS 204) account signatures verified by a native host function; (ii) consensus-enforced cross-account transaction dependencies built on the transaction pool's existing tag graph, together with the removal of fee tips as a priority auction; (iii) on-chain Groth16 verification with an example "private payroll" circuit; and (iv) a conflict-free batch-payment pallet. We correct several claims of v1.3: CRYSTALS-Kyber is a key-encapsulation mechanism and cannot sign transactions, XMSS is a signature scheme rather than a hash function, BLS12-381 SNARKs are *not* quantum-resistant, and topological sorting does not make MEV extraction computationally hard. Finally, we argue where GPUs belong in a Polkadot SDK chain, which is behind deterministic host functions and in off-chain provers, not inside the state transition function, and demonstrate it: batches of ML-DSA signatures are verified on an Apple GPU through Metal, at up to 4.45× the throughput of 12 CPU cores. A GPU validator and a CPU validator stay in consensus over the same blocks.

Contents

1 Introduction **3**

2 Problem Statement **3**

3 Architecture **4**

4 Post-Quantum Accounts **4**
4.1 Choosing the primitives 4
4.2 Account model 5
4.3 Native verification through a host function 6
4.4 Transition period 6
4.5 What is still classical 6

5 DAG-Ordered Transactions **6**
5.1 The transaction pool is already a DAG . . . 6
5.2 `CheckDagParents` . . . 7
5.3 Removing the priority auction . . . 7
5.4 What a DAG does not do . . . 8
5.5 Sealed ordering: order first, reveal later . . . 8

6 Consensus **9**

7 Zero-Knowledge Verification **10**
7.1 Prover/verifier split . . . 10
7.2 Example: private payroll commitment . . . 10
7.3 Security notes and corrections . . . 10
7.4 Post-quantum proofs and signature compression with SP1 . . . 11

8 Parallel Execution **11**
8.1 The payroll pallet . . . 11
8.2 Weights . . . 12

9 GPUs and the Polkadot SDK **12**
9.1 A Metal implementation . . . 13

10 Implementation and Evaluation **14**
10.1 Implementation . . . 14
10.2 Measurements . . . 14
10.3 Throughput, revisited . . . 15

11 Limitations and Roadmap **15**

12 Conclusion **16**

A Corrections to Version 1.3 **18**

1 Introduction

Three pressures shape the next generation of blockchains. Large-scale quantum computers would break the elliptic-curve signatures that protect almost every account today [15, 17]. Block producers routinely reorder, insert and censor transactions to extract value (MEV) [6]. And smart-contract platforms execute one transaction at a time on one core, leaving most of a modern machine idle.

Canggu [21] proposed answers to all three. The design, the name and the original whitepaper are the work of the Bitwyre team listed above, who open-sourced it. This edition is an independent rebuild of their idea. Its purpose is to explore how far the Polkadot SDK can be pushed toward such a design, including onto GPUs, not to endorse every choice of the original, and it says so where measurements disagree with v1.3. It makes the design concrete on the Polkadot SDK [13], a framework that already provides networking, a transaction pool, block authoring and finality, a Wasm runtime with forkless upgrades, and the FRAME pallet system. Building on it lets us focus on what is specific to Canggu, and every claim below is tied to code that runs.

Contributions.

1. A post-quantum account model on Substrate: ML-DSA-65 signatures, accounts derived as $\mathsf{BLAKE2b\text{-}256}(pk)$, and verification through a native host function (§4).
2. `CheckDagParents`, a transaction extension that turns the transaction pool into a cross-account dependency DAG and makes violating a dependency a consensus fault, plus a fee extension that removes tips (§5).
3. A circuit-agnostic Groth16 verifier pallet backed by a native host function, and a private payroll circuit as a worked example (§7).
4. A batch-payment pallet whose *conflict-freedom contract* is what a parallel executor needs (§8).
5. An assessment of GPUs in a Polkadot SDK chain (§9) and an honest list of what is not solved yet (§11).

Status at a glance. Table 1 maps every component of v1.3 to its Polkadot SDK realisation.

2 Problem Statement

- **Quantum vulnerability.** ECDSA, EdDSA and Schnorr signatures (secp256k1, Ed25519, Sr25519) rely on the discrete-logarithm problem, which Shor's algorithm solves in polynomial time [17]. Estimates put the break of a 256-bit curve at a few thousand logical qubits [15]. Hash functions are only affected by Grover's algorithm [7], which halves their effective security.
- **MEV.** When a single party chooses transaction order, it can front-run, back-run and sandwich users. Fee-priority mempools make this an open auction [6].
- **Sequential execution.** Runtimes such as the EVM apply transactions one by one, even when they touch disjoint state.
- **Consensus bottlenecks.** Fast block production, fast finality and decentralisation pull in different directions.

v1.3 component	Polkadot SDK realisation	Status
Dilithium signatures	`CangguSignature::MlDsa65`, host fn `ml_dsa_65_verify`	IMPLEMENTED
Kyber key management	ML-KEM is a KEM, not a signature; reserved for encrypted mempool / transport	FUTURE WORK
XMSS "hashing"	BLAKE2b-256 for hashing; SLH-DSA (FIPS 205) as conservative signature option	PARTIAL
Hybrid (classical + PQ)	Either-or crypto agility: classical variants kept for migration	IMPLEMENTED
DAG mempool	Pool `provides/requires` tags + `CheckDagParents`	IMPLEMENTED
MEV mitigation	Dependency enforcement, no tips, sealed order-then-reveal (§5.5)	PARTIAL
Tower BFT, rotating leader	Aura (round-robin slots, 2 s) + GRANDPA finality	IMPLEMENTED
Sub-second finality	Tuned GRANDPA: 242 ms p50 on localhost (§6)	PARTIAL
ZK-SNARK verification	`pallet-zk-verifier`: Groth16 and post-quantum SP1 STARKs (§7.4)	IMPLEMENTED
Signature compression	One SP1 proof per intent batch replaces all keys and signatures (§7.4)	PARTIAL
GPU proof generation	Off-chain prover; GPU MSM/NTT back-ends (e.g. ICICLE) pluggable	FUTURE WORK
GPU acceleration	Batch ML-DSA verification on Apple Metal behind a host function	IMPLEMENTED
Parallel CPU/GPU contracts	Conflict-free batch payments; parallel executor	PARTIAL
50,000+ TPS	Not supported by measurements with PQ signatures; see §10	REVISED

Table 1: From the v1.3 design to the Polkadot SDK implementation.

3 Architecture

A Polkadot SDK chain has two halves. The *node* (client) handles networking, the transaction pool, consensus and storage, and is native code. The *runtime* is the state transition function, compiled to WebAssembly, stored on-chain and executed by every validator in a sandbox. The runtime can only reach the outside world through *host functions*, a fixed and versioned interface the node implements. Figure 1 shows where each Canggu component lives.

Transaction lifecycle. A wallet signs a transaction with ML-DSA-65 and submits it to any node. The pool validates it by calling into the runtime: the signature is checked by the `ml_dsa_65_verify` host function, `CheckNonce` and `CheckDagParents` emit its `provides/requires` tags, and the no-tip fee extension checks it can pay. The current Aura leader drains the pool in dependency order into a block. Every validator re-executes the block, including the same extension checks, and GRANDPA finalises it.

4 Post-Quantum Accounts

4.1 Choosing the primitives

NIST finalised its first post-quantum standards in August 2024: ML-KEM (FIPS 203, formerly CRYSTALS-Kyber) [10], ML-DSA (FIPS 204, formerly CRYSTALS-Dilithium) [11] and SLH-DSA (FIPS 205, formerly SPHINCS+) [12]. Version 1.3 proposed Kyber for "key management", Dilithium for signing and XMSS for "hashing". Two corrections are needed:

- **ML-KEM cannot authenticate transactions.** A KEM establishes a shared secret between two parties; it has no signing operation. Account keys must be signature keys. In

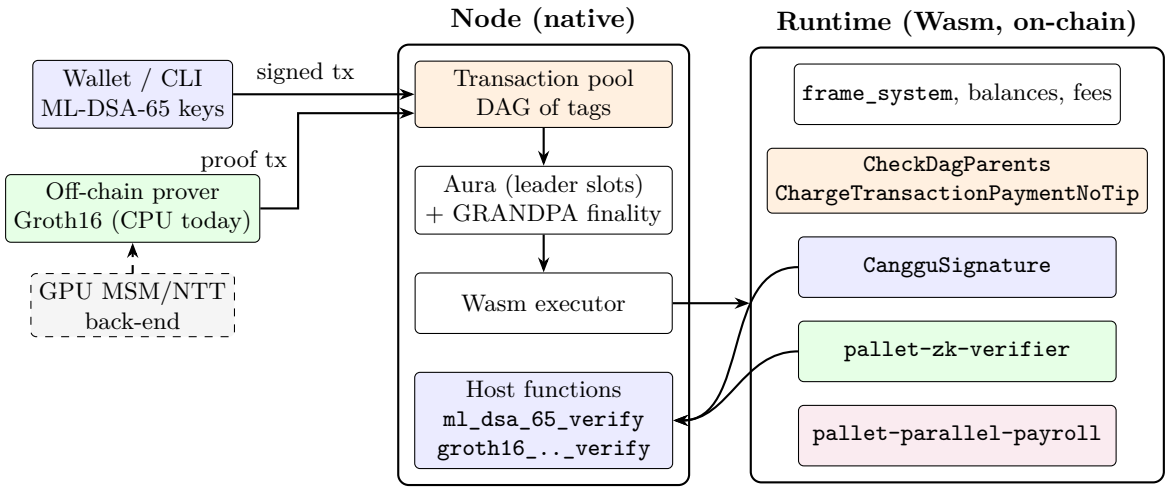


Figure 1: Canggu on the Polkadot SDK. Expensive cryptography is pulled out of the Wasm runtime into native host functions; that interface is also where hardware acceleration can be added without touching consensus logic. Dashed: not yet implemented.

Canggu, ML-KEM is reserved for confidentiality: an encrypted mempool (§11) and hybrid key exchange in the peer-to-peer transport.

- **XMSS is a stateful signature scheme** [5], not a hash function, and reusing a one-time state is catastrophic. That makes it a poor fit for user wallets that may be restored on several devices. Transaction and block hashing in Canggu uses BLAKE2b-256. Grover's algorithm reduces its preimage resistance to 2^{128} quantum operations, which already meets the post-quantum target. Where a hash-based *signature* is wanted for conservatism, the right choice is the stateless SLH-DSA.

Canggu therefore uses **ML-DSA-65** (NIST security category 3) for accounts. Public keys are 1,952 bytes and signatures 3,309 bytes.

4.2 Account model

Substrate identifies accounts by a 32-byte `AccountId32`, and a transaction carries the signer's account plus a signature. For Ed25519 and Sr25519 the account *is* the public key. That cannot work for a 1,952-byte ML-DSA key, so Canggu defines

$$\mathsf{account}(pk) = \mathsf{BLAKE2b\text{-}256}(pk)$$

and the signature variant carries the key itself:

```
pub enum CangguSignature {
    Ed25519(ed25519::Signature),        // index 0  (transition period)
    Sr25519(sr25519::Signature),        // index 1  (transition period)
    Ecdsa(ecdsa::Signature),            // index 2  (transition period)
    MlDsa65 { public: MlDsa65Public, signature: MlDsa65Signature }, // index 3
}

impl Verify for CangguSignature {
    fn verify(&self, msg, signer: &AccountId32) -> bool {
        match self {
            Self::MlDsa65 { public, signature } =>
                public.to_account_id() == *signer
                    && canggu_crypto::ml_dsa_65_verify(&public.0, msg, &signature.0),
            /* classical variants as in sp_runtime::MultiSignature */
```

```
        }
    }
}
```

Binding the key to the account by hash prevents key substitution: a valid signature under a different key fails the first check, at the cost of one hash. Signatures use the FIPS 204 context string `"canggu/tx/v1"` for domain separation, and the payload is Substrate's standard signed payload (call, extensions, spec and transaction version, genesis hash), so replay across chains and runtime versions is prevented exactly as for classical accounts.

4.3 Native verification through a host function

Verification runs natively in the node through a new runtime interface, in the same way Substrate verifies Sr25519:

```
#[runtime_interface]
pub trait CangguCrypto {
    fn ml_dsa_65_verify(public: &[u8], msg: &[u8], signature: &[u8]) -> bool;
    fn groth16_bls12_381_verify(vk: &[u8], inputs: &[u8], proof: &[u8]) -> bool;
}
```

The node registers these functions alongside `sp_io::SubstrateHostFunctions`. The reference implementations use the pure-Rust `ml-dsa` [16] and arkworks [2] crates; ML-DSA-65 verification takes 61 µs (Table 6). Keeping the lattice and pairing arithmetic out of Wasm has two benefits. The node can use native code, SIMD and later dedicated hardware, and the runtime does not have to ship and maintain the cryptographic library. It also defines a narrow, deterministic contract (bytes in, one bit out) that alternative implementations, including GPU ones, can meet (§9).

4.4 Transition period

The classical variants keep existing wallets and tooling working during migration, which v1.3 called a "hybrid" period. We deliberately implement *crypto agility* (each account picks one scheme) rather than requiring both signatures on every transaction. A dual signature is only as strong as its stronger half and costs the size of both. Once the ecosystem has migrated, a runtime upgrade can reject the classical variants. Funds in classical accounts can then only be moved by a final migration transaction or recovery process, which governance must schedule *before* a cryptographically relevant quantum computer exists.

4.5 What is still classical

Account signatures are only one of the places a chain uses elliptic curves. In the reference implementation the Aura block-authoring keys (Sr25519), GRANDPA finality votes (Ed25519) and the libp2p transport handshake (X25519) are still classical. A quantum adversary could therefore forge blocks or finality votes even though user funds are protected. Migrating session keys to ML-DSA requires keystore and consensus-engine support and is the top item on the roadmap.

5 DAG-Ordered Transactions

5.1 The transaction pool is already a DAG

Substrate's transaction pool does not keep a FIFO queue. Each validated transaction declares a set of opaque *tags* it `provides` and a set it `requires`. A transaction is *ready* when every required tag is provided by a transaction that is on-chain or already ready. Otherwise it waits in the

future queue. Block authors drain the ready set in an order consistent with these edges, which is Kahn's algorithm, as proposed in v1.3. The stock `CheckNonce` extension uses this mechanism to chain one account's transactions: transaction (a, n) provides tag $\langle a, n\rangle$ and requires $\langle a, n-1\rangle$.

What is missing is the ability to depend on *another account's* transaction, which is what the v1.3 DEX example needs ("the user's swap references the current state of the pool via a parent transaction").

5.2 `CheckDagParents`

Definition 1. A transaction reference is a pair (a, n) of an account and a nonce. Because nonces are consumed exactly once and in order, (a, n) identifies at most one transaction, and that transaction has executed if and only if $\mathsf{nonce}(a) > n$ in the current state.

`CheckDagParents` is a transaction extension carrying up to eight parent references:

Algorithm 1 `CheckDagParents` (pool validation and block execution)

```
1: function VALIDATE(tx, parents P)                          ▷ in the pool
2:     R ← { encode(a, n) | (a, n) ∈ P, nonce(a) ≤ n }
3:     return VALID(requires = R)          ▷ same encoding as CheckNonce's provides
4: end function
5: function PREPARE(tx, parents P)                           ▷ when the block is executed
6:     if ∃(a, n) ∈ P : nonce(a) ≤ n then
7:         return INVALID(FUTURE)
8:     end if
9:     return OK
10: end function
```

During validation, unmet parents become pool edges. During block execution the same condition is re-checked against state. An author's block builder skips a transaction that fails it, and a block that contains one is invalid and rejected by every honest validator. The extension runs before `CheckNonce` in the extension pipeline, so a transaction rejected for a missing parent has not mutated any state, including the sender's nonce.

Theorem 1 (Dependency safety)**.** *Assume account nonces are only incremented by `CheckNonce`. In any chain accepted by honest validators, if transaction t declares parent (a, n), then the transaction p signed by a with nonce n is executed before t: in an earlier block, or earlier in the same block.*

Proof. Honest validators execute every extrinsic of a block, and a block whose extrinsic fails PREPARE is invalid. So when t executes, $\mathsf{nonce}(a) > n$. Nonces start at 0 and increase by exactly one when, and only when, a's transaction with the current nonce is applied (`CheckNonce` rejects any other nonce). Hence a's transaction with nonce n has been applied, and it is p by uniqueness. □

The reference implementation checks this end to end (§10). A child submitted *before* its parent waits in the pool and is included only after the parent. Executing the child first returns FUTURE.

5.3 Removing the priority auction

In Substrate the fee extension also sets the transaction's *priority*, which grows with its tip. Tips are therefore a sealed-bid auction for position: the main tool an MEV searcher uses to land in front of a victim. Canggu replaces `ChargeTransactionPayment` with `ChargeTransactionPaymentNoTip`, which has an identical encoding but rejects any non-zero tip. Priority then depends only on dispatch class, weight and length, never on payment.

5.4 What a DAG does not do

Version 1.3 argued that MEV extraction becomes computationally infeasible because it requires re-sorting a dense DAG in $\Omega(n^2)$ time. We do not rely on this argument:

1. A topological order is computed in $O(|V| + |E|)$, and the producer needs only *one* profitable order, not all of them. Any linear extension of the DAG is a valid block, and their number is typically exponential, so the producer has more freedom than under nonce-only ordering, not less.

2. Dependencies are declared by users. A victim's transaction that does not name a parent, and most do not, can be front-run by any transaction that is independent of it in the DAG.

3. "Hard to manipulate, easy to verify" does not hold either. Checking that an order is topological is linear, and so is producing a manipulated one.

What the DAG does provide is a *safety* guarantee: dependencies that users state are enforced by consensus. Combined with tip removal, this eliminates intra-dependency reordering and paid priority. Mitigating front-running of independent transactions requires hiding transaction contents until their order is fixed (§5.5).

5.5 Sealed ordering: order first, reveal later

The Rand Protocol whitepaper [20], by Dendi Suhubdy and Anish Mohammed, corrects the same mistake in its own first draft. Hiding contents is not enough, because the party that orders can still order by metadata (its Proposition 14.2). Its Theorem 15.4 eliminates front-running under three conditions: (i) the order is fixed while contents are hidden; (ii) there is a single canonical order that binds the block producer; (iii) what is visible before ordering is uniform. Rand meets (i) with t-of-n threshold encryption [3]. Practical schemes of that kind are pairing-based and not post-quantum, and the running Rand chain does not implement it. Canggu's `pallet-sealed-ordering` meets the three conditions with hash commitments:

1. **Commit** in block N: anyone, typically a relayer, posts

$$c = \mathsf{BLAKE2b}(\texttt{canggu/sealed/v1},\ \mathit{sender},\ \mathit{call},\ \mathit{salt})$$

   and locks a deposit. Commitments are uniform 32-byte values, and a relayer hides the sender (iii).

2. **Reveal** in blocks $N{+}1 \ldots N{+}w$: the sender publishes (call, salt). Because c binds the sender, nobody else can reveal or replay it.

3. **Execute** at the start of block $N{+}w{+}1$: all revealed calls of block N run in increasing order of $\mathsf{BLAKE2b}(c \,\|\, \mathsf{hash}(N))$ (ii). This order does not depend on inclusion or reveal order, and it cannot be computed when committing (i). Unrevealed commitments forfeit their deposit.

Proposition 2. *Let a victim's commitment be included in block N. Any transaction whose author learned the victim's call from its reveal executes after the victim's call.*

Proof. The reveal happens no earlier than block $N{+}1$, so a reacting commitment lands in a block $N' > N$. Block N' settles at $N'{+}w{+}1 > N{+}w{+}1$, after all of block N's calls. Within block N, every commitment was made without knowledge of the victim's call, and positions are fixed by $\mathsf{hash}(N)$, which is unknown when committing. □

Compared with threshold encryption, commit-reveal concedes three things. A sender may decline to reveal, a free option priced by the deposit. A leader may censor reveals within the window. The leader of block N can bias $\mathsf{hash}(N)$, though that only permutes commitments whose contents nobody knows. Back-running is not addressed. On the dev chain, three senders revealed in reverse order and executed in the canonical order, and an unrevealed commitment forfeited its deposit. The pallet's tests check order independence and the proposition above. Anonymous commitments without a relayer could use ring-membership proofs such as Parity's `verifiable` [14], at the cost of reintroducing an elliptic-curve assumption.

6 Consensus

Version 1.3 called for a "leader-verified consensus akin to Solana's Tower BFT". Tower BFT is a PBFT variant that uses Proof-of-History as a clock and exponentially growing vote lockouts [23]. Canggu achieves the same split between fast leader-based production and BFT finality with components that are proven in production:

- **Aura** assigns 2-second slots to authorities round-robin. The slot's leader authors the block, so leader rotation is deterministic and transparent.
- **GRANDPA** [18] finalises chains rather than individual blocks. Validators vote on the highest block they consider final, and a supermajority ($> 2/3$) finalises the chain up to that block.

With Substrate's defaults, blocks finalise about five seconds after they appear. Measurement shows that this is not GRANDPA's message pattern but two conservative parameters. The voting rule BeforeBestBlockBy(2) makes validators vote two blocks behind the best block, which costs two slot times. A voter also waits up to $2g$ before prevoting and $4g$ before precommitting, where g is the gossip duration (333 ms by default). Lowering the vote lag does not affect safety: a block is final only with precommits from more than two-thirds of the voters, and votes for a losing fork finalise the common ancestor instead. Both parameters are exposed as node flags, and the slot length as a compile-time constant.

Slot	Gossip g	Vote lag	Finality p50	p90
2 s	333 ms	2	5 s	6 s
2 s	333 ms	0	1 s	2 s
2 s	50 ms	0	242 ms	288 ms
500 ms	333 ms	2	2 s	3 s
500 ms	50 ms	0	217 ms	293 ms
500 ms	25 ms	0	121 ms	173 ms

Table 2: Time from a block's first appearance to its finalisation, two validators on localhost.

Table 2 is a lower bound: on localhost messages arrive almost instantly. Over a real network g must exceed the message delay between validators, and each round needs about two delays. A validator set with 50–100 ms between members should therefore finalise in well under a second, while a globally distributed one probably will not. v1.3's sub-second claim is reachable with GRANDPA for a geographically compact validator set, and remains to be measured over a real network.

Running as a Polkadot parachain or JAM service [22] instead would buy shared security, at the cost of relay-chain finality latency. It would also cost Canggu its custom host functions (§4, §9.1): relay-chain validators re-execute parachain blocks with Polkadot's standard host interface, so ML-DSA and proof verification would have to run inside the runtime, and the GPU back-end would not apply.

7 Zero-Knowledge Verification

7.1 Prover/verifier split

Proof generation runs off-chain on any hardware. The chain only verifies. `pallet-zk-verifier` is circuit-agnostic:

- `register_verifying_key(vk)` stores a Groth16/BLS12-381 verifying key under $\mathsf{id} = \mathsf{BLAKE2b}(vk)$.
- `submit_proof(id, inputs, proof)` calls the `groth16_bls12_381_verify` host function and, on success, records the statement $\mathsf{BLAKE2b}(\mathsf{id} \,\|\, \mathsf{inputs})$ so other pallets and off-chain consumers can rely on it.

Malformed keys, proofs and non-canonical field elements make the host function return `false` rather than panic. A failed verification still pays its full fee.

Verification cost is independent of circuit size: three pairings (the fourth, $e(\alpha, \beta)$, is precomputed) and a multi-scalar multiplication over the ℓ public inputs. It is therefore $O(\ell)$, which v1.3 called "$O(1)$". The two agree for the small, fixed ℓ of a given circuit.

7.2 Example: private payroll commitment

The bundled circuit connects ZK to Canggu's payroll use case. For a fixed number of recipients N, it proves

$$\exists\, s, a_1, \ldots, a_N : \quad a_i \in [0, 2^{64}), \quad \sum_i a_i = T, \quad \mathsf{MiMC\text{-}MP}(s, a_1, \ldots, a_N) = C$$

for public (T, C). An employer publishes the commitment C to a salary table and later proves that the payroll it funds totals T, without revealing any salary. The commitment uses MiMC [1] with exponent 5 (a permutation of the BLS12-381 scalar field, since $\gcd(5, r-1) = 1$), 110 rounds, in Miyaguchi–Preneel mode. Each round costs three R1CS constraints, each range check 64, and materialising each amount and each intermediate hash one more, so the circuit has

$$330\,(N+1) + 64N + N + (N+1) + 2 \;=\; 396N + 333$$

constraints, e.g. 1,917 for $N = 4$ (Table 7). This is demonstration code: the hash parameters are standard but the implementation has not been audited.

7.3 Security notes and corrections

- **Trusted setup.** Groth16 needs a circuit-specific setup whose randomness ("toxic waste") allows forging proofs. The CLI performs a single-party setup for demonstration. Production keys require a multi-party ceremony, which is secure if one participant is honest.
- **BLS12-381 is not post-quantum.** Version 1.3 described "quantum-resistant curves (e.g. BLS12-381 with NIST-approved tweaks)". No such tweak exists. Pairing-based SNARKs rely on discrete logarithms in groups of roughly 255 bits, which Shor's algorithm breaks. A quantum adversary could *forge* Groth16 proofs, although Groth16's perfect zero-knowledge means it could not extract witnesses from existing proofs. The post-quantum path is hash-based proof systems (STARKs/FRI [4]), and Canggu now has one (§7.4).

7.4 Post-quantum proofs and signature compression with SP1

SP1 [19] proves the execution of ordinary Rust programs compiled to RISC-V. Its *compressed* proofs are STARKs over the KoalaBear field with Poseidon2 Merkle commitments and FRI. They use no pairings or elliptic curves, rely only on hash functions, and need no trusted setup. Canggu verifies them with one more host function, `sp1_v6_verify_compressed`. SP1's verifier does not build for the `wasm32v1-none` target that Polkadot SDK runtimes use. Two programs use it:

- **Payroll.** The statement of §7 written as a short Rust function, with a Keccak commitment instead of MiMC. `pallet-zk-verifier` accepts its proofs next to Groth16 and records the statement (*program*, *public values*).
- **Signature compression.** An ML-DSA-65 verifier runs inside SP1 with Keccak-f delegated to SP1's precompile. It is bit-exact with the RustCrypto implementation on 4,280 native cases and 460 zkVM executions. It proves that every (pk_i, m_i, σ_i) in a batch is valid and commits

$$n \,\|\, \mathsf{Keccak}(ctx) \,\|\, \mathsf{Keccak}\Big(\|_i \,\mathsf{BLAKE2b}(pk_i) \,\|\, \mathsf{Keccak}(m_i)\Big).$$

  `pallet-pq-intents::submit_compressed` receives only $(sender_i, intent_i)$ pairs and the proof. Because accounts are $\mathsf{BLAKE2b}(pk)$, the runtime can rebuild the committed value from the intents and the genesis hash alone, so the proof binds exactly these senders and intents on this chain. No key or signature reaches the block.

Proof size (any program, any batch)	1,272,546 B
Verification (native / through the node's host function)	25.5 ms / ≈40 ms
ML-DSA-65 inside SP1 (with / without Keccak precompile)	0.70 M / 1.83 M cycles
Proving `mldsa-batch`, 4 / 64 / 128 signatures	72 s / 617 s / 1,219 s
Peak memory while proving	15–17 GB
Break-even against raw keys and signatures	≈242 signatures (≈37 min)
Proving `payroll`, 16 / 65,536 amounts	50 s / 141 s

Table 3: SP1 v6.10 on an Apple M4 Pro, CPU prover. Live on a development chain, a five-salary payroll proof took 59 s and an eight-intent compressed batch 113 s to prove. Both verified on chain.

Table 3 gives the economics. A compressed proof costs as much block space as ≈242 ML-DSA keys and signatures, and the CPU prover spends ≈9.4 s per signature. Compression therefore works end to end but is a net loss on a laptop: the eight-intent demonstration batch is 30 times larger than the signatures it replaces. It pays off for batches of hundreds of signatures proven on GPUs (SP1's GPU prover targets CUDA), which is the regime Rand Protocol's pipelined aggregator market [20] is designed for. Two findings matter for any chain adopting SP1. First, the raw verifier accepts several byte strings for one valid proof: it ignores trailing bytes and silently reduces field elements $\geq p$. Canggu's wrapper accepts only the canonical encoding, which matters for anything that hashes or deduplicates proofs. Second, SP1's default prover settings exceeded 33 GB of memory at 16 signatures, so a memory-safe configuration is part of the tooling.

8 Parallel Execution

8.1 The payroll pallet

Version 1.3's flagship example is a payroll: the CPU validates and debits the employer once, and the GPU credits all employees "in parallel". `pallet-parallel-payroll` implements the

semantics and, more importantly, the *conflict-freedom contract* that makes parallel execution legal:

Algorithm 2 `batch_pay`$(payer, [(r_1, v_1), \ldots, (r_N, v_N)])$

1: **require** r_i pairwise distinct and $payer \notin \{r_i\}$ ▷ disjoint write sets
2: $V \leftarrow \sum_i v_i$ (checked) ▷ phase 1: sequential
3: $c \leftarrow \textsc{Withdraw}(payer, V)$ ▷ one balance check, one debit
4: **for all** i **do** ▷ phase 2: data-parallel
5: $(c_i, c) \leftarrow \textsc{Split}(c, v_i)$; $\textsc{Resolve}(r_i, c_i)$
6: **end for**

Proposition 3. *Under the precondition of line 1, the credits of phase 2 commute: every permutation yields the same post-state. The call is atomic (all credits or none) and conserves total issuance.*

Proof sketch. Credit i reads and writes only r_i's account, and the accounts are pairwise distinct and distinct from the payer, so the operations act on disjoint state and commute. Dispatch is transactional, so any failing credit reverts the whole call. The withdrawn imbalance is split exactly into the credits, so no value is created or destroyed. All three properties are covered by the pallet's tests. □

Today the credits are applied in a loop. The runtime is single-threaded Wasm, and each credit is dominated by state-trie reads and writes. The pallet nonetheless guarantees what a parallel executor needs: transactions or sub-operations with *declared, disjoint access sets.*

8.2 Weights

Weights must reflect execution time on reference hardware, because they bound block production time. Version 1.3's "parallel gas" per GPU thread block is only meaningful once execution actually runs in parallel and its cost has been measured. Until then Canggu charges linear weights for batch payments. All weights in the reference implementation are hand-set and must be replaced by benchmarks before production.

9 GPUs and the Polkadot SDK

Can the Polkadot SDK run on GPUs? The useful answer separates the parts of a node.

Not in the state transition function. Every validator must compute bit-identical state roots, so the runtime has to be deterministic, metered and sandboxed. GPU execution fails all three as a general contract platform. Floating point and the order of reductions vary across vendors, drivers and architectures. Kernels cannot be pre-empted or metered per instruction, so gas accounting has no hook. Kernels cannot reach the Merkle-trie state, so every storage access would be a host round-trip. Requiring CUDA would also make one vendor's hardware a consensus dependency and exclude CPU-only validators that must re-execute blocks. The v1.3 claim that 10,000 stablecoin transfers run "in under 1 ms" on a GPU ignores that the work is updating 10,000 trie leaves, not adding 10,000 integers.

Behind host functions: yes, with a contract. A host function is a pure function from bytes to bytes with a CPU reference implementation. A node may compute it on a GPU if it returns *exactly* the same output, which is achievable for integer-only kernels. Good candidates are

batch ML-DSA verification (dominated by number-theoretic transforms), multi-scalar multiplication and NTTs for SNARK/STARK verification, and bulk hashing. The deployment discipline is: a CPU reference implementation defines consensus, the GPU back-end is differentially tested against it, and nodes without a GPU fall back to the CPU. Canggu's `canggu_crypto` interface is designed for this, and §9.1 implements it.

Off-chain proving: the best fit. Proof generation is dominated by MSMs and FFTs that parallelise extremely well, and its output is checked on-chain, so it does not have to be deterministic. GPU libraries such as ICICLE [8] report order-of-magnitude speed-ups for these kernels. This is also how GPUs can raise Canggu's throughput under post-quantum signatures: a GPU prover can aggregate thousands of ML-DSA signatures into one STARK, so blocks carry one proof instead of 3.3 kB per transaction (§10).

Parallel execution: multi-core first. Parallelism inside block execution is a CPU problem: run conflict-free transactions on different cores, using declared access sets such as those of the payroll pallet. JAM [22] takes this further with many independent cores executing PolkaVM code in parallel.

9.1 A Metal implementation

To test how close a Polkadot SDK chain can get to the GPU, Canggu adds a batch host function,

$$\texttt{ml_dsa_65_batch_verify}\colon [(pk_i, m_i, \sigma_i)]_{i=1}^{n} \mapsto [b_i]_{i=1}^{n},$$

and a consumer for it. In `pallet-pq-intents`, users sign transfer *intents* off-chain and a relayer submits up to 640 of them in one extrinsic. The runtime makes a single host call per batch, then executes or rejects each intent independently. The node chooses at start-up where the call runs: one CPU core (the reference that defines the verdicts), all cores, or an Apple GPU via Metal (`-crypto-backend metal`).

Kernels. FIPS 204 verification is split at the point where work becomes fixed-size and data-parallel. A multi-threaded CPU stage decodes keys and signatures, checks hint encodings and the $\|z\|_\infty$ bound, and computes tr, μ and the challenge c. Four integer-only Metal kernels then compute the input NTTs, the matrix expansion fused with $\hat{A}\hat{z}$ (the dominant cost: 30 SHAKE128 rejection-sampled polynomials per signature), the inverse NTT with UseHint and w_1 encoding, and the final SHAKE256 comparison with $\tilde{c}$. Small batches below 64 signatures go to the CPU, because a GPU submission has a fixed latency of about 0.2 ms.

Bit-exactness. A GPU that disagreed with the CPU on a single signature would fork its validator off the network, so correctness is established three ways:

- *Differential tests.* The Metal kernels, a pure-Rust mirror of each kernel, and the RustCrypto `ml-dsa` crate are compared on 5,454 items: 2,504 valid signatures, and 2,950 corrupted ones (bit flips in every field, malformed hints, z at the bound, wrong contexts and lengths). All three agree on every item, both as one batch and as random sub-batches.
- *Self-test.* At node start-up the GPU's UseHint is checked against the CPU for *every* input below q, along with the modular arithmetic and Keccak-f. During development this caught a deliberately planted off-by-one that no real signature exercises.
- *Consensus.* Two validators run with different backends, Metal and a single CPU core. Aura alternates authors, so each batch is verified on the GPU by one validator and on the CPU by the other. Six blocks of 256 intents each, with forgeries included, were authored alternately by both validators and finalised. Both nodes report identical hashes for every block.

Batch	ml-dsa, 1 core	ml-dsa, 12 cores	Metal (wall)	GPU busy	vs 12 cores
1	0.060 ms	0.060 ms	0.330 ms	0.203 ms	0.18×
64	3.77 ms	0.61 ms	0.45 ms	0.23 ms	1.35×
256	15.2 ms	1.95 ms	0.85 ms	0.43 ms	2.29×
1024	61.2 ms	7.52 ms	2.12 ms	1.50 ms	3.55×
4096	246 ms	29.0 ms	6.51 ms	5.59 ms	4.45×

Table 4: Batch ML-DSA-65 verification on an Apple M4 Pro (medians). Metal wall time includes the CPU stage and buffer transfers; GPU busy time is reported by Metal's command-buffer timestamps. At 4,096 signatures the GPU verifies 630k signatures per second, 1.37 µs of GPU time each.

Assessment. The GPU wins for batches of 64 signatures and more, and the advantage grows with batch size (Table 4). Inside a live node, 640 signatures took 8.4 ms of wall time and 4.1 ms of GPU time. That is slower than the warm benchmark, because the batch is copied across the host-function boundary and the GPU was shared with other processes. Weights are still charged at single-core CPU cost, because every validator, GPU or not, must keep up. The GPU therefore buys headroom and lower block-execution latency, not a higher on-chain limit, unless the validator set agrees to assume GPU hardware. Only Metal is implemented. A CUDA back-end would sit behind the same interface and the same differential tests.

10 Implementation and Evaluation

10.1 Implementation

The reference implementation targets `polkadot-stable2609` and Rust 1.93.

Crate	Role
`canggu-primitives`	`CangguSignature`, `CangguSigner`, host functions, dev keyring
`canggu-dag-ordering`	`CheckDagParents`, `ChargeTransactionPaymentNoTip`
`pallet-zk-verifier`	Groth16 verifying-key registry and proof verification
`pallet-parallel-payroll`	Conflict-free batch payments
`canggu-zk`	Private payroll circuit, setup and proving (off-chain)
`pallet-pq-intents`	Batches of ML-DSA-signed intents, one batch host call
`canggu-gpu`	ML-DSA-65 batch verification on Metal, CPU mirror, differential tests
`canggu-runtime`	FRAME runtime wiring all of the above
`canggu-node`	Aura + GRANDPA node with Canggu host functions
`canggu-cli`	ML-DSA wallet, prover, end-to-end demo, benchmarks

Table 5: Crates of the reference implementation.

Unit tests cover each pallet and extension. Runtime integration tests push ML-DSA-signed extrinsics through `Executive`: valid transfers execute, bit-flipped and key-substituted signatures are rejected with BadProof, a child applied before its parent returns Future, and tips are refused. The CLI's `demo` command repeats these checks against a live development node.

10.2 Measurements

All numbers were taken on an Apple M4 Pro (12 cores) with release builds.
Two observations matter for the design. First, the prover/verifier asymmetry is real and large: at $N = 1024$ proving takes $\sim 870\times$ longer than verifying (on 12 cores versus one), and every validator only pays the verifier's 2.3 ms. Second, a careless circuit can scale quadratically. An early version of the gadget kept the running hash as a symbolic linear combination, so every

ML-DSA-65 (pure Rust, one core)	
Sign (deterministic)	≈ 0.75 ms
Verify (host function reference implementation)	61 µs
Public key / signature	1,952 B / 3,309 B
Signed `transfer_keep_alive` extrinsic	5,345 B

Table 6: Post-quantum account costs.

Recipients N	**Constraints**	**Setup**	**Prove**	**Verify**	**Proof**
4	1,917	25 ms	33 ms	2.5 ms	192 B
16	6,669	57 ms	68 ms	2.3 ms	192 B
64	25,677	176 ms	185 ms	2.3 ms	192 B
256	101,709	0.60 s	0.57 s	2.3 ms	192 B
1024	405,837	2.3 s	2.0 s	2.3 ms	192 B

Table 7: Private payroll circuit, Groth16 over BLS12-381 (arkworks). Setup and proving use all cores on the CPU; verification is the single-threaded host function and includes decoding and subgroup-checking the verifying key. Proving cost grows linearly with N; verification does not.

MiMC round re-copied its whole history, and proving $N = 256$ took 119 s instead of 0.57 s. Materialising the hash state and each amount as a witness costs one constraint each and restores linear scaling.

Capacity of the current configuration. A block may contain up to 5 MiB, of which 75 % is available to normal transactions, and 1 s of execution weight in a 2 s slot. With 5.3 kB per ML-DSA transfer, a block holds about 740 transfers, so the chain is *bandwidth-bound* at roughly 370 transfers per second. Signature verification for those transfers takes about 740×61 µs ≈ 45 ms on one core, far from the weight limit.

10.3 Throughput, revisited

An ML-DSA-65 transfer is 5,345 bytes on the wire, against roughly 150 bytes for an Sr25519 transfer. The current configuration is bandwidth-bound at about 370 transfers per second (§10.2). At v1.3's target of 50,000 TPS the chain would have to gossip, store and verify about 265 MB/s of transaction data, before any other overhead. Signature verification itself parallelises across cores and is not the binding constraint. Bandwidth and storage are. We therefore no longer claim 50,000 TPS. Reaching that order of magnitude with post-quantum signatures requires *signature compression*: the leader, or a GPU prover, verifies the block's signatures inside a STARK and replaces them with one proof. This is the strongest argument for the ZK pillar. §7.4 implements it per intent batch and measures the price: a 1.27 MB proof, so compression saves space only above ≈242 signatures, and minutes to hours of proving unless GPUs do it.

11 Limitations and Roadmap

1. **Post-quantum consensus keys.** Move Aura and GRANDPA session keys to ML-DSA. This needs keystore support, larger votes and justifications, and is a prerequisite for calling the chain, not only its accounts, quantum-secure.

2. **Threshold-encrypted mempool.** Replace commit–reveal (§5.5) with encryption to a validator threshold key, which removes the free option and needs no second transaction.

Post-quantum threshold decryption is still a research topic; order-fairness protocols [9] are an alternative.

3. **Post-quantum proofs.** Done with SP1 (§7.4). Retire the Groth16 path and pin the verifier version through host-function versioning.

4. **Windowed signature compression.** Per-batch compression works (§7.4); scale it with Rand's block-level aggregation [20]: GPU aggregators prove a STARK over the ML-DSA verifications of a window of finalised blocks off the critical path, the aggregate that covers the most wins, and proofs are bound to their aggregator.

5. **More GPU back-ends.** CUDA next to Metal (§9.1), and GPU Groth16/STARK verification behind the same differential-testing discipline.

6. **Parallel block execution** using declared access sets.

7. **Benchmarked weights**, a multi-party trusted setup ceremony, and audits.

8. **Retire classical account signatures** by runtime upgrade.

12 Conclusion

Building Canggu on the Polkadot SDK shows that most of its vision fits naturally into an existing, production-grade framework. Post-quantum accounts are a new signature type and a host function. The DAG mempool is a transaction extension on top of a pool that already orders by dependencies. On-chain SNARK verification is a pallet and a host function. Bringing the design into contact with code and measurements also sharpened it. Several v1.3 claims did not survive, among them Kyber-signed wallets, quantum-safe BLS12-381 and MEV resistance through graph complexity. Others became more precise: GPUs belong in provers and behind deterministic host functions, and the ZK pillar is what can make post-quantum signatures affordable at scale.

References

[1] Martin Albrecht, Lorenzo Grassi, Christian Rechberger, Arnab Roy, and Tyge Tiessen. MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In *Advances in Cryptology – ASIACRYPT 2016*, pages 191–219. Springer, 2016.

[2] arkworks contributors. arkworks zksnark ecosystem. https://arkworks.rs, 2025. Version 0.6.

[3] Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Paper 2022/898, 2022. URL https://eprint.iacr.org/2022/898.

[4] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018. URL https://eprint.iacr.org/2018/046.

[5] David A. Cooper, Daniel C. Apon, Quynh H. Dang, Michael S. Davidson, Morris J. Dworkin, and Carl A. Miller. Recommendation for stateful hash-based signature schemes. Special Publication 800-208, NIST, 2020.

[6] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE Symposium on Security and Privacy*, pages 910–927, 2020.

[7] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.

[8] Ingonyama. ICICLE: a gpu library for zero-knowledge acceleration. https://github.com/ingonyama-zk/icicle, 2024.

[9] Mahimna Kelkar, Fan Zhang, Steven Goldfeder, and Ari Juels. Order-fairness for Byzantine consensus. In *Advances in Cryptology – CRYPTO 2020*, pages 451–480. Springer, 2020.

[10] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication FIPS 203, NIST, 2024.

[11] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Federal Information Processing Standards Publication FIPS 204, NIST, 2024.

[12] National Institute of Standards and Technology. Stateless hash-based digital signature standard. Federal Information Processing Standards Publication FIPS 205, NIST, 2024.

[13] Parity Technologies. Polkadot SDK. https://github.com/paritytech/polkadot-sdk, 2026. Release polkadot-stable2609.

[14] Parity Technologies. verifiable: proof of membership of a set with known members. https://github.com/paritytech/verifiable, 2026.

[15] Martin Roetteler, Michael Naehrig, Krysta M. Svore, and Kristin Lauter. Quantum resource estimates for computing elliptic curve discrete logarithms. In *Advances in Cryptology – ASIACRYPT 2017*, pages 241–270. Springer, 2017.

[16] RustCrypto. ml-dsa: Pure rust implementation of ML-DSA (FIPS 204). https://crates.io/crates/ml-dsa, 2026. Version 0.1.1.

[17] Peter W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994.

[18] Alistair Stewart and Eleftherios Kokoris-Kogia. GRANDPA: a byzantine finality gadget. arXiv:2007.01560, 2020. URL https://arxiv.org/abs/2007.01560.

[19] Succinct Labs. SP1: a zero-knowledge virtual machine for RISC-V. https://github.com/succinctlabs/sp1, 2026. Version 6.10.0.

[20] Dendi Suhubdy and Anish Mohammed. Randprotocol: Confidential arbitrary computations. Whitepaper, Draft 3, http://randprotocol.org/papers/randprotocol.pdf; full node by D. Suhubdy, https://github.com/randprotocol/fullnode, 2026.

[21] Dendi Suhubdy, Tejveer Singh, Mitul Dhawan, and Maldini Sonkeng. Canggu blockchain: A quantum-secure, MEV-resistant blockchain for parallel computing. Whitepaper v1.3, https://github.com/agustinustheo/canggu-blockchain-whitepaper, 2025.

[22] Gavin Wood. JAM: Join-accumulate machine. a mostly-coherent trustless supercomputer. The Gray Paper, 2024. URL https://graypaper.com.

[23] Anatoly Yakovenko. Solana: A new architecture for a high performance blockchain. https://solana.com/solana-whitepaper.pdf, 2018.

A Corrections to Version 1.3

v1.3 statement	**Correction**
Users generate key pairs with Kyber for wallet operations.	ML-KEM is a KEM. Wallet keys are ML-DSA (FIPS 204).
Transactions are hashed with XMSS.	XMSS is a stateful signature scheme. Hashing uses BLAKE2b-256 (128-bit post-quantum preimage security).
ZK-SNARKs on "quantum-resistant curves (e.g. BLS12-381)".	Pairing-based SNARKs are broken by Shor's algorithm. Use STARKs for post-quantum soundness.
DAG reordering costs $\Omega(n^2)$, making MEV infeasible.	Topological sorting is $O(\|V\| + \|E\|)$ and any linear extension is valid. DAGs enforce declared dependencies only.
FIFO sorting is $O(n)$ / $O(n \log n)$ (both stated).	Neither is relevant: Substrate pools order by priority and tags.
Groth16 verification is $O(1)$.	$O(\ell)$ in the number of public inputs, independent of circuit size.
Solana executes transactions sequentially in a single thread.	Solana's Sealevel executes non-conflicting transactions in parallel. Canggu's distinction is GPU acceleration, not parallelism as such.
50,000+ TPS with Dilithium signatures.	Not supported. Requires signature compression (§10).
Sub-second finality with Tower-BFT-style consensus.	Aura + GRANDPA does it once the vote lag and gossip duration are tuned (localhost measurement; network latency pending).